S2C2 - Solucion Ecuaciones No Lineales

```
# Computación Avanzada y Analítica

# Solución de ecuaciones no lineales

En este notebook aprenderemos a implementar los principales métodos numéricos para encont
rar raíces de funciones no lineales utilizando Python.

## Objetivos

- Comprender el problema de búsqueda de raíces.
- Implementar métodos de intervalo.
- Implementar métodos abiertos.
- Comparar velocidad de convergencia.
- Aplicar los algoritmos a problemas físicos reales.
```

Importación de librerías

```
import numpy as np
import matplotlib.pyplot as plt
```

1. ¿Qué significa encontrar una raíz?

```
Una raíz de una función es un valor

$$
x=\alpha
$$

tal que

$$
f(\alpha)=0
$$

Muchos problemas físicos pueden escribirse de esta forma.

Ejemplos:

- Movimiento orbital (Ecuación de Kepler)
- Pozo cuántico finito
- Ecuación de Van der Waals
- Osciladores
- Sistemas eléctricos
```

Código

```
def f(x):
    return x**3 - x - 2

x = np.linspace(-2,3,400)

plt.figure(figsize=(8,4))
plt.plot(x,f(x))
plt.axhline(0,color='k')
plt.grid()
plt.show()
```

Encontrando visualmente la raíz

```
Observe la gráfica.

Preguntas:

- ¿Dónde parece estar la raíz?
- ¿Existe una sola?
- ¿Cómo podría encontrarse con mayor precisión?
```

2. Métodos de intervalo

```
Los métodos de intervalo utilizan el Teorema del Valor Intermedio.

Si

$$
f(a)f(b)<0
$$

entonces existe al menos una raíz dentro del intervalo.
```

Código

```
a=1
b=2

print(f(a))
print(f(b))
print(f(a)*f(b))
```

Visualización

```
plt.figure(figsize=(8,4))
plt.plot(x,f(x))
plt.scatter([a,b],[f(a),f(b)],s=80)
plt.axhline(0,color='k')
plt.grid()
plt.show()
```

3. Método de Bisección

Markdown

```
La idea es muy sencilla.

1. Se calcula el punto medio.

2. Se evalúa la función.

3. Se conserva únicamente el intervalo donde ocurre el cambio de signo.

Este proceso reduce el intervalo a la mitad en cada iteración.
```

Código completo

```
def biseccion(f,a,b,tol=1e-6,max_iter=100):

    historia=[]

    for i in range(max_iter):

        c=(a+b)/2

        historia.append(c)

        if abs(f(c))<tol:
            break

        if f(a)*f(c)<0:
            b=c
        else:
            a=c

    return c,historia
```

Ejecución

```
raiz,h=biseccion(f,1,2)

print(raiz)
```

Visualización de la convergencia

```
errores=np.abs(np.array(h)-raiz)

plt.semilogy(errores,'o-')
plt.xlabel("Iteración")
plt.ylabel("Error")
plt.grid()
plt.show()
```

4. Método de Falsa Posición

Markdown

```
En lugar del punto medio, este método utiliza la intersección de la recta secante con el eje x.

Esto puede acelerar la convergencia dependiendo de la forma de la función.
```

Código

```
def falsa_posicion(f,a,b,tol=1e-6,max_iter=100):

    historia=[]

    for i in range(max_iter):

        c=(a*f(b)-b*f(a))/(f(b)-f(a))

        historia.append(c)

        if abs(f(c))<tol:
            break

        if f(a)*f(c)<0:
            b=c
        else:
            a=c

    return c,historia
```

Comparación

```
r,h=falsa_posicion(f,1,2)

print(r)
```

5. Métodos abiertos

```
A diferencia de los métodos anteriores, estos algoritmos no necesitan un intervalo.

Su rapidez suele ser mayor, pero dependen mucho de la condición inicial.
```

6. Newton-Raphson

Markdown

```
El método utiliza la recta tangente

$$
x_{n+1}=x_n-\frac{f(x_n)}{f'(x_n)}
$$

Su convergencia es cuadrática cuando se cumplen las hipótesis matemáticas.
```

Código

```
def df(x):
    return 3*x**2-1
```

Implementación

```
def newton(f,df,x0,tol=1e-8,max_iter=50):

    historia=[]

    x=x0

    for i in range(max_iter):

        historia.append(x)

        xn=x-f(x)/df(x)

        if abs(xn-x)<tol:
            return xn,historia

        x=xn

    return x,historia
```

Ejemplo

```
r,h=newton(f,df,1.5)
```

```
print(r)
```

7. Método de la Secante

Markdown

```
La secante evita calcular la derivada.

En su lugar aproxima

$$
f'(x)
$$

mediante diferencias finitas.
```

Código

```
def secante(f,x0,x1,tol=1e-8,max_iter=50):

    historia=[]

    for i in range(max_iter):

        historia.append(x1)

        x2=x1-f(x1)*(x1-x0)/(f(x1)-f(x0))

        if abs(x2-x1)<tol:
            return x2,historia

        x0=x1
        x1=x2

    return x2,historia
```

Comparación entre métodos

Markdown

```
Compararemos:

- Número de iteraciones
- Error
- Tiempo
```

Código

```
from time import perf_counter
```

Ejemplo

```
metodos={}
```

(Aquí se ejecutan los cuatro métodos y se almacena el número de iteraciones, el tiempo de ejecución y el error final para compararlos en una tabla.)

Comparación gráfica

```
# Bisección
rb, hb = biseccion(f, 1, 2)

# Falsa posición
rf, hf = falsa_posicion(f, 1, 2)

# Newton-Raphson
rn, hn = newton(f, df, 1.5)

# Secante
rs, hs = secante(f, 1, 2)
```

```
plt.figure(figsize=(8,5))

plt.semilogy(np.abs(np.array(hb)-rb),label="Bisección")
plt.semilogy(np.abs(np.array(hf)-rf),label="Falsa posición")
plt.semilogy(np.abs(np.array(hn)-rn),label="Newton")
plt.semilogy(np.abs(np.array(hs)-rs),label="Secante")

plt.legend()
plt.grid()
plt.show()
```

8. Aplicación física: Ecuación de Kepler

Markdown

```
Resolveremos

$$
E-e\sin(E)-M=0
$$

utilizando Newton-Raphson.
```

```
Esta ecuación aparece en mecánica orbital para calcular la posición de un planeta en una órbita elíptica.
```

Código

```
e=0.6
M=1
```

```
def kepler(E):
    return E-e*np.sin(E)-M

def dkepler(E):
    return 1-e*np.cos(E)
```

```
newton(kepler,dkepler,M)
```

9. Aplicación física: Pozo cuántico finito

Markdown

```
En este problema aparecen funciones tangente con asíntotas.

Por esta razón es más seguro utilizar un método de intervalo como la bisección.
```

(Aquí se define una función simplificada del pozo cuántico y se aplica bisección en un intervalo que no cruce las asíntotas, permitiendo al estudiante explorar diferentes estados ligados.)

10. Aplicación física: Ecuación de Van der Waals

Markdown

```
La ecuación de Van der Waals conduce a un polinomio cúbico con hasta tres raíces reales.

Cada raíz tiene un significado físico distinto:

- líquido
- vapor
- estado inestable
```

(Se implementa la ecuación para valores específicos de (P), (T), (a) y (b), y se utiliza bisección en distintos intervalos para identificar cada raíz.)

11. Ejercicios

```
## Ejercicio 1

Encuentre la raíz de
```

```
$$
x^3-4x+1=0
$$

utilizando:

- Bisección
- Newton
- Secante

Compare el número de iteraciones.

---

## Ejercicio 2

Cambie la condición inicial del método de Newton.

¿Qué ocurre cuando la condición inicial está muy lejos de la raíz?

---

## Ejercicio 3

Grafique el error absoluto en escala logarítmica para los cuatro métodos.

¿Puede identificar visualmente cuál converge más rápido?

---

## Ejercicio 4

Implemente un criterio de parada basado en:

$$
|x_{n+1}-x_n|
$$

y compárelo con el criterio

$$
|f(x_n)|.
$$

---

## Ejercicio 5 (Reto)
```

```
Utilice `scipy.optimize` (`bisect`, `newton` y `root_scalar`) para resolver el mismo prob
lema y compare el rendimiento con sus implementaciones.
```
